

Ex.No:	
Date:	

AIM:

ALGORITHM:

CODE:

```
def isPrime(n):
```

```
    if n <= 1:
```

```
        return False
```

```
    for i in range(2, int(n**0.5) + 1):
```

```
        if n % i == 0:
```

return False

231401102

```
    return True

p = int(input("Enter the first number (p): "))
q = int(input("Enter the second number (q): "))

if isPrime(p):
    print(p, "is a Prime Number.")
else:
    print(p, "is NOT a Prime Number.")

if isPrime(q):
    print(q, "is a Prime Number.")
else:
    print(q, "is NOT a Prime Number.")

if isPrime(p) and isPrime(q):
    print("\nBoth numbers are prime.")
    print("Proceed with RSA Key Generation.")
else:
    print("\nInvalid Prime Number(s).")
    print("RSA Key Generation Aborted.")
```

OUTPUT:

```
Enter the first number (p): 17
Enter the second number (q): 23
17 is a Prime Number.
23 is a Prime Number.
```

```
Both numbers are prime.
Proceed with RSA Key Generation.
```

RESULT:

Ex.No:	
Date:	

AIM:

ALGORITHM:

CODE:

```
def find_gcd(a, b):
```

```
    if a < b:
```

```
        a, b = b, a
```

```
print("\nSteps of Euclidean Algorithm:")
```

```
while b != 0:
```

```
    remainder = a % b
```

```

    print(f' {a} ÷ {b} = Quotient {a//b}, Remainder {remainder}')

    a = b

    b = remainder

    return a

print("=== Secure Communication Key Validation ===")

parameter1 = int(input("Enter the first encryption parameter: "))
parameter2 = int(input("Enter the second encryption parameter: "))

gcd = find_gcd(parameter1, parameter2)

print("\nGreatest Common Divisor (GCD) =", gcd)

if gcd == 1:

    print("\nThe encryption parameters are COPRIME.")

    print("Secure communication can be established.")

else:

    print("\nThe encryption parameters are NOT COPRIME.")

    print("Communication setup rejected.")

    print("Choose different encryption parameters.")

```

OUTPUT:

```

=== Secure Communication Key Validation ===
Enter the first encryption parameter: 35
Enter the second encryption parameter: 12

Steps of Euclidean Algorithm:
35 ÷ 12 = Quotient 2, Remainder 11
12 ÷ 11 = Quotient 1, Remainder 1
11 ÷ 1 = Quotient 11, Remainder 0

Greatest Common Divisor (GCD) = 1

The encryption parameters are COPRIME.
Secure communication can be established.

```

RESULT:

Ex.No:	
Date:	

AIM:

ALGORITHM:

231401102

CODE:

```
print("=====")
print("    MODULAR ARITHMETIC CALCULATOR")
print("=====")

a = int(input("Enter the First Integer (a): "))
b = int(input("Enter the Second Integer (b): "))
m = int(input("Enter the Modulus (m): "))
if m <= 0:
    print("Modulus must be greater than 0.")
else:
    while True:
        print("\n===== MENU =====")
        print("1. Modular Addition")
        print("2. Modular Subtraction")
        print("3. Modular Multiplication")
        print("4. Modular Exponentiation")
        print("5. Exit")
        choice = int(input("Enter your choice (1-5): "))
        if choice == 1:
            result = (a + b) % m
            print("\nModular Addition")
            print(f'({a} + {b}) mod {m} = {result}')
        elif choice == 2:
            result = (a - b) % m
            print("\nModular Subtraction")
            print(f'({a} - {b}) mod {m} = {result}')
        elif choice == 3:
```

```

        result = (a * b) % m

        print("\nModular Multiplication")

        print(f'({a} × {b}) mod {m} = {result}')

    elif choice == 4:

        result = pow(a, b, m)

        print("\nModular Exponentiation")

        print(f'({a}^{b}) mod {m} = {result}')

    elif choice == 5:

        print("\nExiting the Program...")

        Break

    else:

        print("\nInvalid Choice! Please select between 1 and 5.")

```

OUTPUT:

```

=====
MODULAR ARITHMETIC CALCULATOR
=====
Enter the First Integer (a): 15
Enter the Second Integer (b): 7
Enter the Modulus (m): 11

===== MENU =====
1. Modular Addition
2. Modular Subtraction
3. Modular Multiplication
4. Modular Exponentiation
5. Exit
Enter your choice (1-5): 1

Modular Addition
(15 + 7) mod 11 = 0

===== MENU =====
1. Modular Addition
2. Modular Subtraction
3. Modular Multiplication
4. Modular Exponentiation
5. Exit
Enter your choice (1-5): 3

```

Modular Multiplication
 $(15 \times 7) \bmod 11 = 6$

===== MENU =====

1. Modular Addition
2. Modular Subtraction
3. Modular Multiplication
4. Modular Exponentiation
5. Exit

Enter your choice (1-5): 4

Modular Exponentiation
 $(15^7) \bmod 11 = 5$

===== MENU =====

1. Modular Addition
2. Modular Subtraction
3. Modular Multiplication
4. Modular Exponentiation
5. Exit

Enter your choice (1-5): 5

Exiting the Program...

RESULT:

Ex.No:	
Date:	

AIM:

ALGORITHM:

CODE:

```
print("=====")
print("  RSA Prime Validation using Fermat's Test")
print("=====")
a = int(input("Enter the value of a: "))
```

```

p = int(input("Enter the candidate prime number (p): "))
if p <= 1:
    print("\nInvalid Input!")
    print("The candidate number p must be greater than 1.")
else:
    result = pow(a, p - 1, p)
    print("\nResult of Fermat Test")
    print(f'({a}^{p-1}) mod {p} = {result}')
    if result == 1:
        print("\nThe number satisfies Fermat's Little Theorem.")
        print("It is a probable prime.")
        print("Candidate is suitable for further RSA key generation.")
    else:
        print("\nThe number does not satisfy Fermat's Little Theorem.")
        print("It is composite.")
        print("Candidate is NOT suitable for RSA key generation.")

```

OUTPUT:

```

=====
  RSA Prime Validation using Fermat's Test
=====
Enter the value of a: 2
Enter the candidate prime number (p): 17

Result of Fermat Test
(2^16) mod 17 = 1

The number satisfies Fermat's Little Theorem.
It is a probable prime.
Candidate is suitable for further RSA key generation.

```

RESULT:

Ex.No:	
Date:	

AIM:

ALGORITHM:

CODE:

```
import math
```

```
a = int(input("Enter the value of a: "))
```

```
n = int(input("Enter the value of n: "))
```

```
phi = 0
```

```
for i in range(1, n + 1):
```

```
    if math.gcd(i, n) == 1:
```

```

    phi += 1
print("\nEuler's Totient Function")
print("φ(", n, ") =", phi)
if math.gcd(a, n) == 1:
    print(a, "and", n, "are coprime.")
    result = pow(a, phi, n)
    print("\nEuler's Theorem Verification")
    print(f'{a}^{phi} mod {n} = {result}')
if result == 1:
    print("Euler's Theorem is Verified.")
else:
    print("Euler's Theorem is NOT Verified.")
else:
    print(a, "and", n, "are not coprime.")
    print("Euler's Theorem cannot be applied.")

```

OUTPUT:

```

Enter the value of a: 3
Enter the value of n: 10

Euler's Totient Function
φ( 10 ) = 4
3 and 10 are coprime.

Euler's Theorem Verification
3^4 mod 10 = 1
Euler's Theorem is Verified.

```

RESULT: